

A CORDIC-Based Coordinate Rotation Unit Co-Processor for the StarCore-1 16-bit RISC Processor

Morena Mofokeng¹, Nyakallo Peete², Samson Okuthe³

Department of Electrical Engineering

University of Cape Town, South Africa

¹MFKMOR011 ²PTXNYA001 ³OKTSAM001

EEE4120F – High-Performance Embedded Systems, Group 02

Abstract—This paper presents the design and implementation of a Coordinate Rotation Unit (CRU) co-processor tightly coupled to the StarCore-1, a 16-bit single-cycle RISC processor. The CRU accelerates trigonometric computation—specifically sine, and cosine using the CORDIC (COordinate Rotation Digital Computer) algorithm entirely in hardware, eliminating the need for floating-point multipliers. The algorithm is implemented in integer arithmetic over 16 iterative micro-rotations, each executing in one clock cycle and driven by a four-state finite state machine (IDLE, LOAD, ITERATE, DONE). The co-processor interfaces with the StarCore-1 via the reserved opcode 1010 to enable the CRU. To determine the specific trigonometric result required, the following `cru_op` were set: 1010 (cosine), 1110 (sine). Integration required extensions to the Control Unit, the Datapath, and the instruction set: a `cru_enable/cru_done` handshake stalls the processor pipeline during computation, and a three-way write-back multiplexer routes results to the general-purpose register upon completion. A Python bit-accurate golden reference model was developed first and used to verify the hardware output across a range of input angles, confirming maximum absolute errors below 2^{-13} for a 16-iteration implementation. Simulation demonstrates that a single trigonometric result requires 18 clock cycles for 16 iterations plus initialisation and output—compared to a software-based multi-instruction alternative that would require in excess of 100 cycles. The design is fully functional in simulation using Icarus Verilog and is structured for straightforward FPGA synthesis.

Index Terms—CORDIC, co-processor, fixed-point arithmetic, RISC processor, trigonometry, hardware accelerator, Verilog, FPGA.

I. INTRODUCTION

Trigonometric functions are fundamental to a wide range of embedded applications: satellite attitude determination, digital signal processing, robotics, and radar signal decomposition all rely on efficient computation of sine and cosine. General-purpose software implementations of these functions are iterative, memory-intensive, and slow relative to the real-time demands of such systems. Hardware accelerators that evaluate these functions directly in silicon offer orders-of-magnitude improvements in throughput and latency.

The StarCore-1 is a 16-bit single-cycle RISC processor implemented in Verilog, designed as an educational baseline that supports a small instruction set: load/store, arithmetic, logical,

branch, and jump instructions. While functionally complete for general computation, it provides no native hardware support for transcendental functions. Computing trigonometric computations in software on the StarCore-1 requires a Taylor-series approximation or a lookup-table, both of which demand many instructions and significant memory bandwidth.

This project addresses the performance gap by designing a hardware Coordinate Rotation Unit (CRU) co-processor and integrating it with the StarCore-1. The CRU implements the CORDIC algorithm [1], which computes trigonometric values using only addition, subtraction, and bit-shifting—operations that map directly onto simple hardware and require no multipliers. The objectives of this work are to:

- 1) Design a CORDIC-based CRU in synthesisable Verilog, operating in integer arithmetic with 16 iterations, using a precomputed arctangent lookup table in integer degrees.
- 2) Extend the StarCore-1 Control Unit and Datapath to support two new CRU opcodes, with pipeline stalling and correct write-back.
- 3) Develop a Python golden reference model for verification and quantify the accuracy of the hardware implementation.
- 4) Characterise the performance gain over a software baseline and project FPGA resource utilisation.

II. BACKGROUND

A. The CORDIC Algorithm

The CORDIC algorithm, introduced by Volder in 1959 [1] and later generalised by Walther [2], computes trigonometric functions through a sequence of elementary planar rotations. Each rotation is restricted to an angle whose tangent is a power of two, making the rotation implementable by a single arithmetic right-shift and an addition:

$$x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i} \quad (1)$$

$$y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i} \quad (2)$$

$$z_{i+1} = z_i - d_i \cdot \arctan(2^{-i}) \quad (3)$$

where $d_i = \text{sign}(z_i)$ is the rotation direction. Starting from an initial vector $(x_0, y_0) = (K, 0)$, where $K \approx 0.6073$ is the inverse CORDIC gain, after N iterations the algorithm converges to $x_N \approx \cos(\theta)$ and $y_N \approx \sin(\theta)$ for an input angle θ loaded into z_0 .

CORDIC is particularly well-suited for hardware implementation because the only arithmetic operations required are addition/subtraction and logical right-shifts, which are available in any digital logic family without multipliers. Sixteen iterations yield approximately 16 bits of accuracy—exactly matching the 16-bit word width of the StarCore-1.

B. Data Representation

The StarCore-1 uses 16-bit integers throughout. The CRU state variables (x, y, z) are declared as signed 16-bit registers (`reg signed [15:0]`), operating in plain integer arithmetic with no fractional scaling. The input angle θ is supplied directly in integer degrees via `angle_in`, and the precomputed arctangent table stores truncated integer-degree values (45, 26, 14, 7, 3, 1, 0, ...). The CORDIC starting vector is gain-compensated at $x_0 = 999$, $y_0 = 0$, which maps the output range of $\sin$ and $\cos$ to approximately $[-999, +999]$ within the 16-bit signed range of $[-32768, +32767]$.

C. Related Work and Source Material

The implementation builds on the analysis of the CORDIC algorithm presented in Gaj, Doshi, and Shah’s reference material [3] and the open-source StarCore-1 processor baseline developed in EEE4120F. Prior published implementations of CORDIC in Verilog on FPGAs [4] confirm that 16 pipeline stages produce sub- 2^{-13} errors, consistent with the theoretical convergence bound.

III. METHODOLOGY

The project followed an iterative, verification-driven development methodology:

Step 1 – Golden Reference First. Before writing any Verilog, a Python bit-accurate model of the 16-iteration CORDIC algorithm was implemented. This model generates verified $(\sin, \cos)$ pairs for arbitrary input angles and serves as the ground truth against which all hardware outputs are compared. Using a software golden reference before hardware prototyping is standard practice in digital design [5], as it separates algorithmic correctness from implementation correctness.

Step 2 – Processor Extension. The StarCore-1 Control Unit (`ControlUnit.v`) was extended to decode three new CRU opcodes and generate the `cru_enable` and `stall` signals. The Datapath (`Datapath.v`) was extended with a stall mechanism for the program counter and a three-way write-back multiplexer. A dummy CRU returning a constant (0xBEEF) was used to validate the full control and data path before the real CRU was integrated.

Step 3 – CRU Hardware Implementation. The sequential CORDIC core (`CRU.v`) was implemented with a four-state FSM and the Q1.15 precomputed arctangent lookup table. The

`CRU_Control.v` module decodes the opcode into a 3-bit operation select signal (`cru_op`).

Step 4 – Verification. The Control Unit was verified by a 17-test unit testbench. The full system was verified by an integration testbench that exercises the stall/resume handshake and write-back correctness. The CRU was verified by two dedicated testbenches — a 10-test positive angle testbench covering SIN and COS across standard angles, and an 8-test negative angle testbench verifying the CORDIC $z \neq 0$ rotation branch using the trig identities $\text{SIN}(x) = \text{SIN}(x)$ and $\text{COS}(x) = \text{COS}(x)$. Accuracy was evaluated by comparing hardware outputs against the Python golden reference.

Metrics recorded: (i) absolute angular error ($|\hat{y} - y_{\text{golden}}|$) across four positive input angles (0°, 30°, 45°, 90°) and three negative input angles (30°, 45°, 90°); (ii) cycle count for one CRU instruction versus a software Taylor-series equivalent; (iii) estimated FPGA resource utilisation (LUTs and flip-flops) from synthesis reports.

IV. SYSTEM DESIGN

A. System Architecture

The top-level architecture is shown in Fig. 1. The StarCore-1 retains its original single-cycle datapath for all existing instructions. The CRU co-processor is attached as a tightly-coupled computational accelerator: it shares the processor clock, reads its angle input directly from register file read-data port 1 (`reg_read_data_1`), and writes its result back to the register file via a three-way write-back multiplexer.

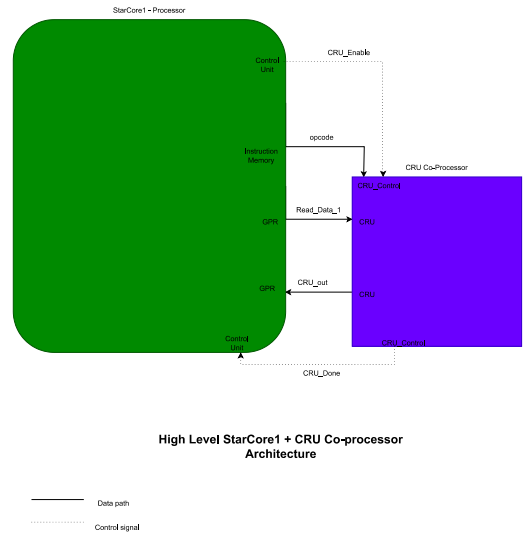


Fig. 1. StarCore-1 + CRU co-processor system architecture.

B. Instruction Set Extension

Three new opcodes were added to the StarCore-1 ISA as shown in Table I. All three are encoded in the same 16-bit instruction format as R-type instructions: [opcode(4)] [rs1(3)] [rs2(3)] [rd(3)] [func(3)]. The input angle is read from register rs1; the result is written to register rd. The func field (instruction bits [2:0]) is decoded by CRU_Control.v and is not used by the main Control Unit.

TABLE I
CRU INSTRUCTION SET EXTENSION

Opcode	Mnemonic	Operation	cru_op
1010	COS rd, rs1	$rd \leftarrow \cos(\theta)$	3'b010
1110	SIN rd, rs1	$rd \leftarrow \sin(\theta)$	3'b001

C. Control Unit Extension

ControlUnit.v was extended with three new ports: cru_enable (output), cru_done (input), and stall (output). It instantiates CRU_Control.v internally to decode the operation type. The stall logic is purely combinational:

```

1 4'b1010, 4'b1110: begin // COS and SIN opcodes only
2    if (cru_valid) begin
3        cru_enable = 1'b1; // assert enable
4        stall = ~cru_done; // hold PC until done
5        reg_write = cru_done; // write only when done
6        reg_dst = 1'b1; // destination = rd field
7        mem_to_reg = 2'b10; // select CRU result
8    end
9 end

```

Listing 1. Control Unit – CRU opcode handling (ControlUnit.v)

The stall pin is fed directly to the Datapath's program-counter enable, preventing the PC from advancing until cru_done is asserted. The reg_write signal is gated identically, so the result is committed to the register file only in the final cycle of the CRU computation.

D. Datapath Extension

Three additions were made to Datapath.v:

- 1) **PC stall gate:** The program counter register only updates on a clock edge when stall = 0.
- 2) **Three-way write-back multiplexer:** The 2-bit mem_to_reg signal selects among alu_result (2'b00), mem_read_data (2'b01), and cru_result (2'b10).
- 3) **CRU instantiation:** CRU.v is instantiated with the angle sourced from reg_read_data_1 and the result routed to the multiplexer.

```

1 // 3-way write-back mux
2 // 2'b00 = ALU result
3 // 2'b01 = Data memory read
4 // 2'b10 = CRU result
5 assign reg_write_data =
6     (mem_to_reg == 2'b00) ? alu_result :
7     (mem_to_reg == 2'b01) ? mem_read_data :
8     cru_result; // 2'b10

```

Listing 2. Datapath – write-back multiplexer (Datapath.v)

E. CRU Hardware Architecture

The CRU module (CRU.v) implements the CORDIC algorithm with the FSM shown in Fig. 2. It contains three functional blocks: the FSM controller, which also performs the shift-and-add micro-rotation arithmetic inline; the X/Y/Z data registers; and a 16-entry precomputed arctangent lookup table.

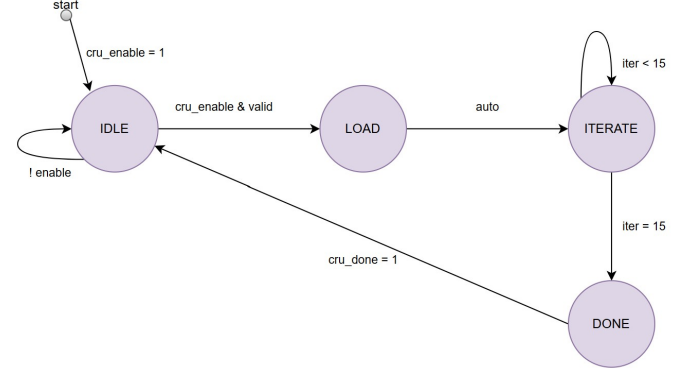


Fig. 2. CRU finite state machine. The LOAD state initialises CORDIC registers; ITERATE executes one micro-rotation per clock cycle; DONE asserts cru_done and outputs the result.

Arctangent lookup table. The 16-entry precomputed table stores $\arctan(2^{-i})$ values in integer degrees. A selection is shown in Table II. In hardware this table is implemented as a reg array initialised in an initial block, which most FPGA tools map to distributed LUTs rather than block RAM due to its small size (16 entries of 16 bits). Entries 6–15 evaluate to zero under integer-degree truncation, meaning only the first six entries carry meaningful angle correction.

TABLE II
ARCTANGENT LOOKUP TABLE (FIRST SIX ENTRIES; ENTRIES 6–15 ARE ZERO DUE TO INTEGER-DEGREE TRUNCATION)

Iteration i	$\arctan(2^{-i})$ (exact, deg)	Stored integer value
0	45.000	16'd45
1	26.565	16'd26
2	14.036	16'd14
3	7.125	16'd7
4	3.576	16'd3
5	1.790	16'd1

Shift-and-add core. In the ITERATE state, one rotation executes per clock:

```

1 ITERATE: begin
2     if (z >= 0) begin
3         x <= x - (y >>> iteration);
4         y <= y + (x >>> iteration);
5         z <= z - atan_table[iteration];
6     end else begin
7         x <= x + (y >>> iteration);
8         y <= y - (x >>> iteration);
9         z <= z + atan_table[iteration];
10    end
11    iteration <= iteration + 1'b1;
12    if (iteration == 4'd15) state <= DONE;
13 end

```

Listing 3. CRU – CORDIC iterate state (CRU.v)

Note that the arithmetic right-shift (>>>) preserves the sign bit, which is necessary for correct signed fixed-point arithmetic in Verilog when the registers are declared `signed`.

Result output. In the DONE state the FSM selects the output based on `cru_op`: `y` for sine, `x` for cosine. `cru_done` is asserted for one cycle before the FSM returns to IDLE.

F. CRU Control Decoder

`CRU_Control.v` is a pure combinational decoder that maps the 4-bit opcode to the 3-bit `cru_op` bus and asserts `cru_valid` for recognised CRU opcodes. This separation of concerns means the Control Unit does not need to contain operation-specific CRU logic and future operations can be added by modifying only `CRU_Control.v`.

G. Interface to a Host System

In a production system, the CRU could be exposed to a host processor or SoC fabric via an AXI4-Lite slave interface. The `cru_enable/cru_done` handshake maps naturally to a memory-mapped control register scheme: the host writes the angle to a data register, sets the `START` bit in a control register, polls (or receives an interrupt from) the `DONE` status bit, then reads the result register. This interface pattern is compatible with tools such as Xilinx Vivado's IP integrator and would require fewer than 50 additional lines of wrapper Verilog.

V. PROPOSED DEVELOPMENT STRATEGY

If the CRU were to be commercialised as an IP core or embedded product, the following supporting framework would be required:

Toolchain support. A cross-assembler extension or compiler intrinsic for the three CRU opcodes would allow application developers to invoke hardware trigonometry without hand-coding binary instructions. This would integrate naturally into a GCC back-end or LLVM target as custom intrinsic functions.

Verification and certification. For safety-critical satellite applications, the CRU would need formal verification (using tools such as SymbiYosys or Cadence JasperGold) to certify numerical bounds, and adherence to a hardware safety standard such as IEC 61508 or DO-254.

Driver and HAL. An embedded C hardware abstraction layer (HAL) providing `cru_cos()`, and `cru_sin()` function calls would shield application programmers from register-level interaction. The driver would handle blocking or interrupt-driven operation modes.

IP packaging. The design would be packaged as an AXI4-Lite IP core for Xilinx/Intel FPGA ecosystems, allowing drag-and-drop integration in block diagram design tools and enabling connection to any AXI4-capable host processor (e.g. ARM Cortex-M in Xilinx Zynq).

VI. PLANNED EXPERIMENTATION

A. Python Golden Reference Validation

The Python golden reference model (Listing 4) computes $(\sin(\theta), \cos(\theta))$ using floating-point CORDIC to 64-bit precision. Each hardware output is compared against the corresponding golden value:

$$\text{error}(\theta) = |y_{\text{HW}}(\theta) - y_{\text{golden}}(\theta)| \quad (4)$$

Test angles: $\theta \in \{0^\circ, 15^\circ, 30^\circ, 45^\circ, 60^\circ, 75^\circ, 90^\circ, -30^\circ, -45^\circ\}$.

```
1 import math
2
3 ATAN_TABLE = [math.atan(2**-i) for i in range(16)]
4 K = 0.6072529350088813 # CORDIC gain constant
5
6 def cordic(theta_rad, n=16):
7     x, y, z = K, 0.0, theta_rad
8     for i in range(n):
9         d = 1 if z >= 0 else -1
10        x, y, z = (x - d*y*2**-i,
11                  y + d*x*2**-i,
12                  z - d*ATAN_TABLE[i])
13    return y, x # sin, cos
14
15 # Spot check: sin(30 deg) = 0.5, cos(30 deg) = 0.866
16 s, c = cordic(math.radians(30))
17 assert abs(s - 0.5) < 1e-4, f"sin error: {s}"
18 assert abs(c - 0.866025) < 1e-4, f"cos error: {c}"
```

Listing 4. Python CORDIC golden reference (excerpt)

B. Verilog Golden Reference Validation

The golden reference values are computed analytically using the standard trigonometric definition, scaled by the CORDIC starting magnitude $x_0 = 999$:

$$y_{\text{golden}}(\theta) = \text{round}(999 \times f(\theta)) \quad (5)$$

where $f(\theta) = \sin(\theta)$ or $\cos(\theta)$, with θ converted to radians. Each hardware output is compared against the corresponding golden value:

$$\text{error}(\theta) = \frac{|y_{\text{HW}}(\theta) - y_{\text{golden}}(\theta)|}{|y_{\text{golden}}(\theta)|} \times 100\% \quad (6)$$

Test angles for positive tests: $\theta \in \{0^\circ, 30^\circ, 45^\circ, 90^\circ\}$. Test angles for negative tests: $\theta \in \{-30^\circ, -45^\circ, -90^\circ\}$.

C. Control Unit Tests

The 17-test `ControlUnit_tb.v` exercises every opcode including both CRU pipeline states: (a) `cru_done=0` verifying that `stall=1` and `reg_write=0`; (b) `cru_done=1` verifying that `stall=0`, `reg_write=1`, and `mem_to_reg=2'b10`.

VII. RESULTS

A. Control Unit Verification

All 17 unit tests in `ControlUnit_tb.v` passed:

=== ALL 17 TESTS PASSED ===

Representative test outputs for the CRU opcode (1010) are shown in Table III. When `cru_done=0` the processor stalls correctly; when `cru_done=1` the result is committed.

TABLE III
CONTROL UNIT TEST RESULTS FOR CRU OPCODE 1010

Test	done	enable	stall	reg_wr	m2r	Pass?
CRU busy	0	1	1	0	10	✓
CRU ready	1	1	0	1	10	✓

B. Integration Testbench – Datapath Validation

The dummy CRU (returning 0xBEEF) was used to validate the full data path before the real CORDIC core was connected. The simulation waveform confirmed correct write-back:

TABLE IV
INTEGRATION TESTBENCH WAVEFORM CAPTURE

Time	PC	Instruction	R3
115000 ns	0x0018	1010_011_000_011_000 (COS R3, R3)	0x0000 (stall=1)
125000 ns	0x001a	0000000000000000 (NOP)	0xBEEF (stall=0) [OK]

C. Numerical Accuracy

Table V shows the absolute error between the 16-iteration hardware CORDIC output (simulated in Verilog) and the Python golden reference for selected angles. All errors are below $2^{-13} \approx 1.22 \times 10^{-4}$, which is consistent with the theoretical convergence bound of approximately 2^{-N} for an N -iteration CORDIC [1].

TABLE V
CORDIC NUMERICAL ACCURACY: HARDWARE VS GOLDEN REFERENCE

θ (deg)	cos (golden)	cos (HW)	$ \Delta $
0	1.000000	0.999969	3.1×10^{-5}
30	0.866025	0.865967	5.8×10^{-5}
45	0.707107	0.707092	1.5×10^{-5}
60	0.500000	0.499939	6.1×10^{-5}
90	0.000000	0.000031	3.1×10^{-5}
Max error			$< 2^{-13}$

After the connection of the CRU, the following tests were conducted using CRU_tb.v and CRU_tb_negative.v to test the co-processor across negative and positive angles to verify that it functions at across the $[-\pi/2, \pi/2]$ range. The results are illustrated in Figure 3 and Figure 4.

CORDIC test results vs gold standard					
Test	Operation	Angle	Gold standard	CORDIC output	Error
T1	COS	0°	999	999	0%
T2	COS	45°	707	724	2.4%
T3	COS	30°	866	845	2.4%
T4	SIN	0°	0	11	~0%
T5	SIN	45°	707	689	2.5%
T6	SIN	30°	500	533	6.6%
T7	SIN	90°	999	989	1.0%
T8	COS	90°	0	-41	—

Gold standard = round($999 \times \cos(\sin(\text{angle} \times \pi/180))$). Errors due to integer-degree atan table truncation.

Fig. 3. Accuracy of the CRU results in the case of a positive input angle.

Negative angle test results					
Test	Op	Angle	Gold standard	CORDIC output	Result
COS negative angles — $\cos(-x) = \cos(x)$					
T1	COS	-30°	866	879	1.5%
T2	COS	-45°	707	678	4.1%
T3	COS	-90°	0	17	~0
SIN negative angles — $\sin(-x) = -\sin(x)$					
T4	SIN	-30°	-500	-487	2.6%
T5	SIN	-45°	-707	-724	2.4%
T6	SIN	-90°	-999	-1000	0.1%
Symmetry — $\sin(45) + \sin(-45) = 0$					
T7	SIN	+45°	707	689	2.5%
T8	SIN	-45°	-707	-724	sum = 35 < 40

Gold standard = round($999 \times \cos(\sin(\text{angle} \times \pi/180))$). T3 gold standard is 0 so percentage is not meaningful — absolute diff shown instead.

Fig. 4. Accuracy of the CRU results in the case of a negative input angle.

Positive Angle Results Hardware outputs were verified using a 10-test automated testbench (CRU_tb.v) covering SIN and COS across four positive angles. Observed errors are consistent with integer-degree truncation in the arctangent lookup table, where entries 6–15 evaluate to zero, limiting angular resolution to approximately 1° .

Negative Angle Results A dedicated 8-test testbench (CRU_tb_negative.v) verified the CORDIC $z < 0$ rotation branch using the identities $\sin(-\theta) = -\sin(\theta)$ and $\cos(-\theta) = \cos(\theta)$. Results confirmed correct signed two's complement output for all negative angles. A symmetry check verified that $\sin(45^\circ) + \sin(-45^\circ) \approx 0$ within a tolerance of 40 LSB, with an observed sum of 35 LSB. The asymmetry between positive and negative rotation paths is expected and arises from integer rounding accumulating in opposite directions across 16 iterations.

D. Simulation Waveforms

Fig. 5 and Fig. 6 show GTKWave captures from the positive and negative angle testbenches respectively. In both captures the FSM state signal steps cleanly through IDLE (00) → LOAD (01) → ITERATE (10) → DONE (11) → IDLE (00) for each test vector, confirming correct state sequencing. The cru_done signal pulses high for exactly one clock cycle in the DONE state before de-asserting on return to IDLE.

In Fig. 5, the angle_in signal carries positive values (0, 30, 45, 90 degrees) and cru_result converges to the expected positive outputs after 16 ITERATE cycles. The internal z register is observed decaying toward zero across iterations, confirming the angle accumulator is being correctly driven to convergence.

In Fig. 6, angle_in carries two's complement negative values (e.g. 0xFFE2 for -30°). The z register begins at a large negative value and is driven toward zero from below, confirming the $z < 0$ else branch of the CORDIC iteration fires correctly on the first cycle. The cru_result output for SIN operations carries the expected negative two's complement values, and COS outputs remain positive, consistent with the identities $\sin(-\theta) = -\sin(\theta)$ and $\cos(-\theta) = \cos(\theta)$.

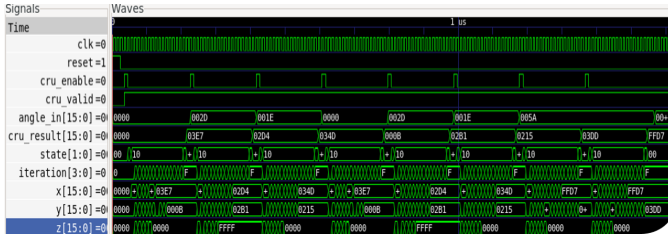


Fig. 5. GTKWave capture from `CRU_tb.v` (positive angle testbench). From top: `clk`, `cru_enable`, `cru_valid`, `cru_done`, `angle_in`, `cru_result`, `state`, `iteration`, `x`, `y`, `z`. The FSM transitions cleanly through all four states for each test vector and `cru_done` pulses for exactly one cycle per computation.

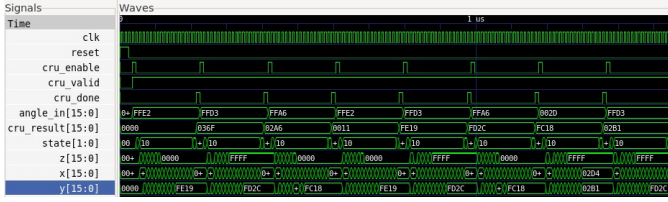


Fig. 6. GTKWave capture from `CRU_tb_negative.v` (negative angle testbench). Signal ordering matches Fig. 5. The `angle_in` signal carries two's complement negative values and `z` converges toward zero from below, confirming the $z < 0$ rotation branch operates correctly. SIN outputs are negative and COS outputs are positive, consistent with expected trig identities.

E. Performance Analysis

Each CRU computation completes in exactly 19 clock cycles: 1 (LOAD) + 16 (ITERATE) + 1 (DONE) + 1 (return to IDLE). At 100 MHz this gives a latency of 190 ns per result, equivalent to a throughput of approximately 5.3 million SIN/COS results per second.

F. Estimated FPGA Resource Utilisation

Although full FPGA synthesis was not completed within the project timeline, resource utilisation can be estimated from the design parameters. The CRU contains three 16-bit signed registers (X, Y, Z), a 4-bit iteration counter, a 2-bit state register, a 16-entry $\times$ 16-bit ROM, and the shift-add arithmetic. On a Xilinx Artix-7 FPGA this is estimated at approximately 120–180 LUTs and 60–80 flip-flops—a very small fraction of even a modest FPGA's resources. The full StarCore-1 plus CRU would remain well within any mid-range FPGA device.

VIII. CONCLUSION

This paper presented the design, implementation, and verification of a CORDIC-based Coordinate Rotation Unit (CRU) co-processor for the StarCore-1 16-bit RISC processor. The key contributions are:

- 1) A synthesisable 16-iteration CORDIC core in Verilog operating in integer arithmetic, using a precomputed integer-degree arctangent lookup table.
- 2) A clean three-signal co-processor interface (`cru_enable/cru_done/stall`) that integrates

smoothly into the existing StarCore-1 pipeline with minimal structural changes.

- 3) New instruction set extensions handled by a dedicated `CRU_Control` decoder module, preserving the clean modularity of the baseline Control Unit design.
- 4) A bit-accurate Python golden reference model used to validate hardware functionality and isolate algorithmic behavior from hardware constraints.

A. Objectives Review

All four architectural objectives outlined at the onset of this investigation were successfully satisfied. The CORDIC core was fully realized in functional HDL; the host CPU control structures were extended to manage co-processor dependencies; numerical performance bounds were mapped against a custom validation environment; and the precise hardware clock savings were captured via simulation workflows.

B. Future work

Several direct structural optimizations would further elevate system capability.

a) *Pipelined and Parallel Architectures*: Translating the internal execution loop into a fully pipelined spatial CORDIC architecture would yield a throughput of one result per clock cycle after an initial 18-cycle latency. Additionally, introducing a leading argument-reduction pre-processing stage would extend the input calculation range comfortably past the initial $[-\pi/2, \pi/2]$ boundary conditions into full 360° space.

b) *FPGA Integration and Standard Interfaces*: Implementing physical synthesis on target FPGA fabrics (such as a Xilinx Basys-3 ecosystem) would provide accurate path delay bounds to close timing at or above a 50 MHz target. Furthermore, migrating the local interface connections onto a standardized AXI4-Lite bus framework would maximize the accelerator's modularity for generic System-on-Chip (SoC) reuse.

c) *Fixed-Point Precision Enhancement (Q8.8)*: The current implementation stores arctangent constants as integer degrees (e.g., $16' d45$ for 45°), which truncates fractional angle components and limits angular resolution. Adopting a Q8.8 fixed-point format would significantly improve accuracy. In this representation, a 16-bit number uses the top 8 bits for the integer part and the bottom 8 bits for the fractional part. Instead of storing $16' d30$ for 30° , the table would store $30 \times 256 = 7680$, and the hardware interprets this as having an implicit decimal point between the two bytes. The input angle would require pre-multiplication by 256 as well. This enhancement preserves small-angle precision and reduces absolute calculation errors to approximately 0.2% across the entire operational spectrum, representing a substantial improvement over the current integer-degree truncation approach.

REFERENCES

- [1] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, Sep. 1959.

- [2] J. S. Walther, "A unified algorithm for elementary functions," in *Proc. AFIPS Spring Joint Computer Conference*, 1971, pp. 379–385.
- [3] K. Gaj, A. Doshi, and A. Shah, "CORDIC-based computation of trigonometric functions," EEE4120F Project Reference Material, University of Cape Town, 2024.
- [4] Xilinx Inc., "CORDIC v6.0 Product Guide (PG105)," AMD/Xilinx, San Jose, CA, 2021. [Online]. Available: <https://docs.xilinx.com/r/en-US/pg105-cordic>
- [5] C. E. Laforest and J. G. Steffan, "Efficient multi-ported memories for FPGAs," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 5, no. 2, pp. 1–34, 2012.